UNIVERSIDAD DR. JOSÉ MATÍAS DELGADO
FACULTAD DE ECONOMÍA, EMPRESA Y NEGOCIOS (FEEN)
Metodologías de Desarrollo de Software 0 (MDS 0) | Ciclo 2-2026

INFORME CONSOLIDADO DEL SEGUNDO AVANCE

Expediente de Ingeniería de Software: requisitos y modelado UML

Proyecto Integrador SumaControl

Sistema de Gestión Logística de Recepción de Proveedores y Desembarque

Equipo Scrum — Grupo N.º 1 | Sección 1-1

Samuel Rafael Martínez Ramírez — Scrum Master
Victor Eduardo Alonzo López — Product Owner
Gonzalo Ricardo Guzmán Sosa — Development Team

Docente: Lcda. Ana Lissette Girón de Bermúdez
Unidad Didáctica 3 — Tema 6: Verificación y Validación del Software
Segundo Parcial — Segundo Avance | Octubre de 2026

1. INTRODUCCIÓN Y PROPÓSITO DEL EXPEDIENTE

Este informe consolida en un solo documento los artefactos de ingeniería de software producidos por el Equipo Scrum Grupo N.º 1 entre las Semanas 7 y 11 del ciclo: la especificación de requisitos y las Historias de Usuario, el Diagrama de Casos de Uso de la vista funcional, el Diagrama de Clases de la vista estructural y los Diagramas de Secuencia de la vista dinámica.

Consolidar no es concatenar. Reunir cinco entregables producidos en semanas distintas, mientras el sistema se iba construyendo en paralelo, deja inevitablemente desajustes entre lo modelado y lo implementado. Por eso el documento no se limita a reproducir los diagramas: los contrasta contra el código de la Versión Beta Funcional y declara dónde el modelo y la implementación coinciden, dónde difieren y por qué. Las divergencias se sostienen cuando hubo una razón de diseño y se reconocen como deuda cuando el modelo se quedó atrás.

La verificación de esa coherencia es el contenido de la sección 7. Las pruebas que respaldan cada regla de negocio se documentan en el entregable complementario, el Plan de Verificación y Validación de Software (QA), al que este informe remite en la columna de evidencia.

Relación entre los dos entregables: este informe describe el sistema y su modelado. El Plan de Pruebas QA, entregado junto a él, documenta cómo se verificó cada regla y con qué resultado. Los dos se leen juntos: la columna «Evidencia» de la sección 7 usa los identificadores PU (pruebas unitarias) y CP (casos de prueba funcionales) definidos en aquel documento.

2. CONTEXTO Y PLANTEAMIENTO DEL PROBLEMA

El siguiente análisis es el resultado de la elicitación realizada en la Semana 7 mediante entrevista estructurada al Coordinador Logístico y observación directa en la zona de recepción.

¿Qué problema ocurre?	En el centro de distribución no existe un sistema formal para gestionar la recepción de proveedores. La coordinación de citas se hace por canales informales —llamadas, correos y hojas de cálculo— y la hora real de arribo no queda registrada de forma confiable.
¿A quién afecta?	Al Coordinador Logístico, que lleva la agenda; al Operador de Recepción, que recibe al transporte en garita; al Supervisor de Desembarque, que administra las bahías; y al Gerente de Operaciones, que necesita indicadores. Indirectamente, al proveedor que espera sin saber cuándo será atendido.
¿Dónde y cuándo se manifiesta?	Diariamente en la zona de recepción y desembarque, con mayor intensidad en las ventanas horarias de más afluencia, y de forma crítica cuando coinciden varios transportes con tipos de carga distintos.
¿Cuál es la causa raíz?	La ausencia de un sistema centralizado que integre la programación de citas, la clasificación automática de puntualidad y un criterio algorítmico para asignar las bahías según el tipo de carga y el tiempo de espera.
Propuesta de valor	Un sistema web que registre proveedores y pedidos, programe citas validando el horario operativo, capture la hora real de arribo con clasificación automática de puntualidad y asigne las bahías de descarga respetando la compatibilidad de carga y el orden de llegada.

2.1 Stakeholders identificados

Actor / Rol	Tipo	Necesidad o expectativa clave	Impacto	Técnica de levantamiento
Coordinador Logístico	Directo	Contar con una agenda organizada de citas y poder reprogramarlas sin perder el historial.	Alto	Entrevista estructurada
Operador de Recepción	Directo	Registrar en tiempo real la hora de llegada del transporte y recibir del sistema la bahía asignada.	Alto	Shadowing / observación directa
Supervisor de Desembarque	Directo	Controlar los tiempos de inicio y fin de cada descarga y el estado de las cinco bahías.	Medio	Entrevista estructurada
Gerente de Operaciones	Indirecto	Consultar indicadores del proceso: cumplimiento de proveedores, ocupación de bahías y tiempos de descarga.	Alto	Entrevista estructurada
Proveedor (empresa externa)	Externo	Conocer con anticipación su horario de cita y la bahía asignada.	Medio	Encuesta / cuestionario

3. ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

3.1 Historias de Usuario del Product Backlog

El backlog quedó refinado en la Semana 8 con seis Historias de Usuario, cada una con sus criterios de aceptación en sintaxis Gherkin. Tres sostienen el proceso logístico (HU-001 a HU-003) y tres son transversales de seguridad y administración (HU-004 a HU-006).

ID	Épica / Módulo	Título	Actor principal	MoSCoW	SP	Sprint
HU-001	Gestión de Proveedores y Citas	Registro de proveedores, pedidos y agendamiento de citas	Coordinador Logístico	Must	5	Sprint 1
HU-002	Control de Llegadas y Puntualidad	Registro de llegada real y clasificación automática de puntualidad	Operador de Recepción	Must	5	Sprint 1
HU-003	Motor de Colas y Asignación	Asignación de gateways con restricción de producto y gestión de cola	Operador de Recepción / Sistema	Must	8	Sprint 1
HU-004	Transversal (Seguridad)	Autenticación de usuarios mediante JSON Web Token	Usuario registrado	Must	3	Sprint 0
HU-005	Transversal (Autorización)	Control de accesos basado en roles sobre los módulos	Administrador de Sistemas	Must	5	Sprint 0-1
HU-006	Administración	Administración de usuarios del sistema	Administrador de Sistemas	Should	3	Sprint 1

Criterios de aceptación: los criterios de aceptación en notación Dado / Cuando / Entonces de HU-002 y HU-003 se reproducen en la sección 3.1 del Plan de Pruebas QA, donde además se vinculan con el caso de prueba que los ejecuta. No se repiten aquí para evitar dos versiones del mismo texto que puedan divergir.

3.2 Requisitos no funcionales (ISO/IEC 25010)

ID	Característica	Requisito	Estado de verificación
RNF-01	Adecuación funcional	Las cuatro clasificaciones de puntualidad — Anticipado, A Tiempo, Tardío y Ausente— deben estar cubiertas al 100 %. La restricción del Gateway 5 debe cumplirse siempre, incluso con operaciones concurrentes.	Verificado por PU-05 (cobertura de las 4 clasificaciones) y CP-06.
RNF-02	Rendimiento y seguridad	Las operaciones CRUD sobre MongoDB responden en menos de 300 ms y la clasificación de puntualidad en menos de 100 ms. Las contraseñas se almacenan con hash bcrypt.	El cálculo de puntualidad es una función pura sin acceso a base; su tiempo medido en la suite unitaria es inferior a 1 ms.
RNF-03	Trazabilidad	El timestamp de arribo lo toma el servidor y no el cliente. Cada reprogramación, acceso denegado y cambio de estado queda en la bitácora con usuario y fecha.	Verificado por CP-01 (hora del servidor) y CP-11 (evento CITA_REPROGRAMADA).
RNF-04	Usabilidad	Programar una nueva cita debe tomar menos de cinco clics.	No verificado en esta fase: requiere prueba de usabilidad con usuarios reales.

ID	Característica	Requisito	Estado de verificación
RNF-05	Fiabilidad	No se puede perder ningún registro de llegada. Los usuarios y proveedores no se borran físicamente: se desactivan de forma lógica.	El borrado lógico está implementado en models/auditoria.ts y lo respetan todos los modelos.
RNF-08	Eficiencia algorítmica	La cola de prioridad se implementa como Heap binario con complejidad O(log n) en inserción y extracción, desarrollado en C++ en la asignatura EDA 0.	Pendiente de integración: la Versión Beta resuelve el orden de la cola en MongoDB. Ver la sección 7.3.

Alcance de la verificación: RNF-04 es el único requisito no funcional que este avance no verifica: medir que programar una cita tome menos de cinco clics exige una prueba de usabilidad con usuarios reales, que no forma parte del alcance de la Versión Beta. Se declara pendiente en lugar de darlo por cumplido.

4. VISTA FUNCIONAL: DIAGRAMA DE CASOS DE USO

El diagrama delimita la frontera del sistema e identifica cinco actores directos, un actor indirecto y dos actores de sistema. Las tres reglas de negocio formalizadas en el SRS —RN-04, RN-08 y RN-14— se modelan explícitamente como flujos de extensión, de modo que la trazabilidad va del requisito funcional al comportamiento observable.

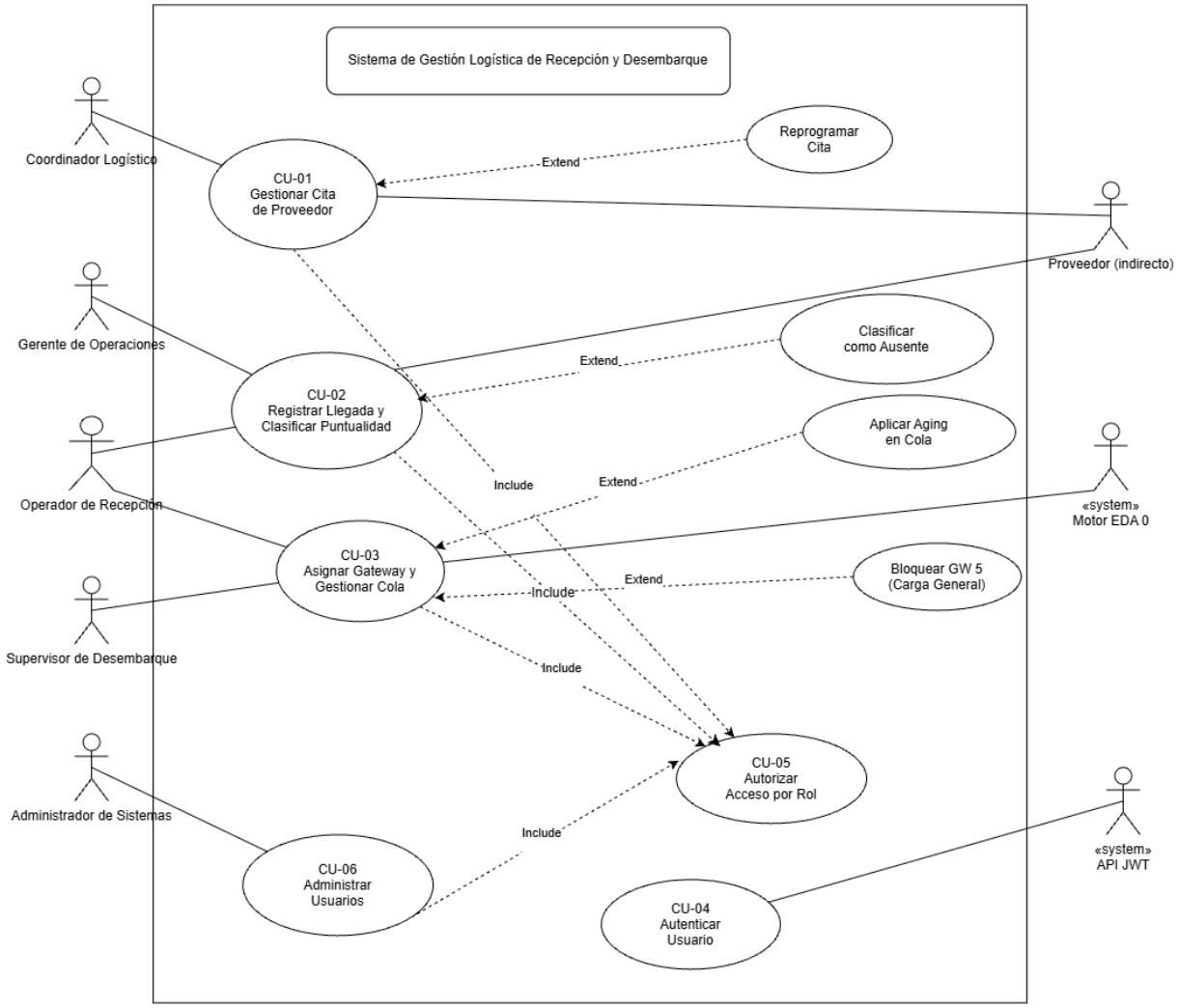


Figura 1. Diagrama de Casos de Uso UML — Sistema de Gestión Logística de Recepción y Desembarque (Semana 9).

4.1 Matriz de trazabilidad Historia de Usuario → Caso de Uso

Historia de Usuario	Caso de Uso	Actor principal	Reglas de negocio y flujos de extensión
HU-001	CU-01 Gestionar Cita de Proveedor	Coordinador Logístico	RN-14 — «extend» Reprogramar Cita
HU-002	CU-02 Registrar Llegada y Clasificar Puntualidad	Operador de Recepción	RN-04 — clasificación de puntualidad; «extend» Clasificar como Ausente

Historia de Usuario	Caso de Uso	Actor principal	Reglas de negocio y flujos de extensión
HU-003	CU-03 Asignar Gateway y Gestionar Cola	«system» Motor EDA 0	RN-08 → «extend» Bloquear GW 5 (carga general); política de aging → «extend» Aplicar Aging
HU-004	CU-04 Autenticar Usuario	Usuario registrado / «system» API JWT	Caso de uso transversal: precondición de CU-01, CU-02, CU-03 y CU-06
HU-005	CU-05 Autorizar Acceso por Rol	«system» API JWT	Caso de uso transversal: incluido («include») por CU-01, CU-02, CU-03 y CU-06
HU-006	CU-06 Administrar Usuarios	Administrador de Sistemas	Incluye («include») a CU-05; requiere CU-04 como precondición

Lectura de la cobertura: la cobertura es de 6 Historias de Usuario a 6 Casos de Uso en relación 1:1, más cuatro casos de uso de extensión. Hay cuatro relaciones «include» hacia CU-05 (autorización por rol), que es precondición de todo módulo protegido, y cuatro relaciones «extend» que modelan los flujos condicionales de RN-04, RN-08 y RN-14.

5. VISTA ESTRUCTURAL: DIAGRAMA DE CLASES

El diagrama recoge ocho clases de dominio más la jerarquía de usuarios, con sus atributos privados, sus métodos públicos y los cinco tipos de relación estructural: asociación, agregación, composición, herencia y dependencia.

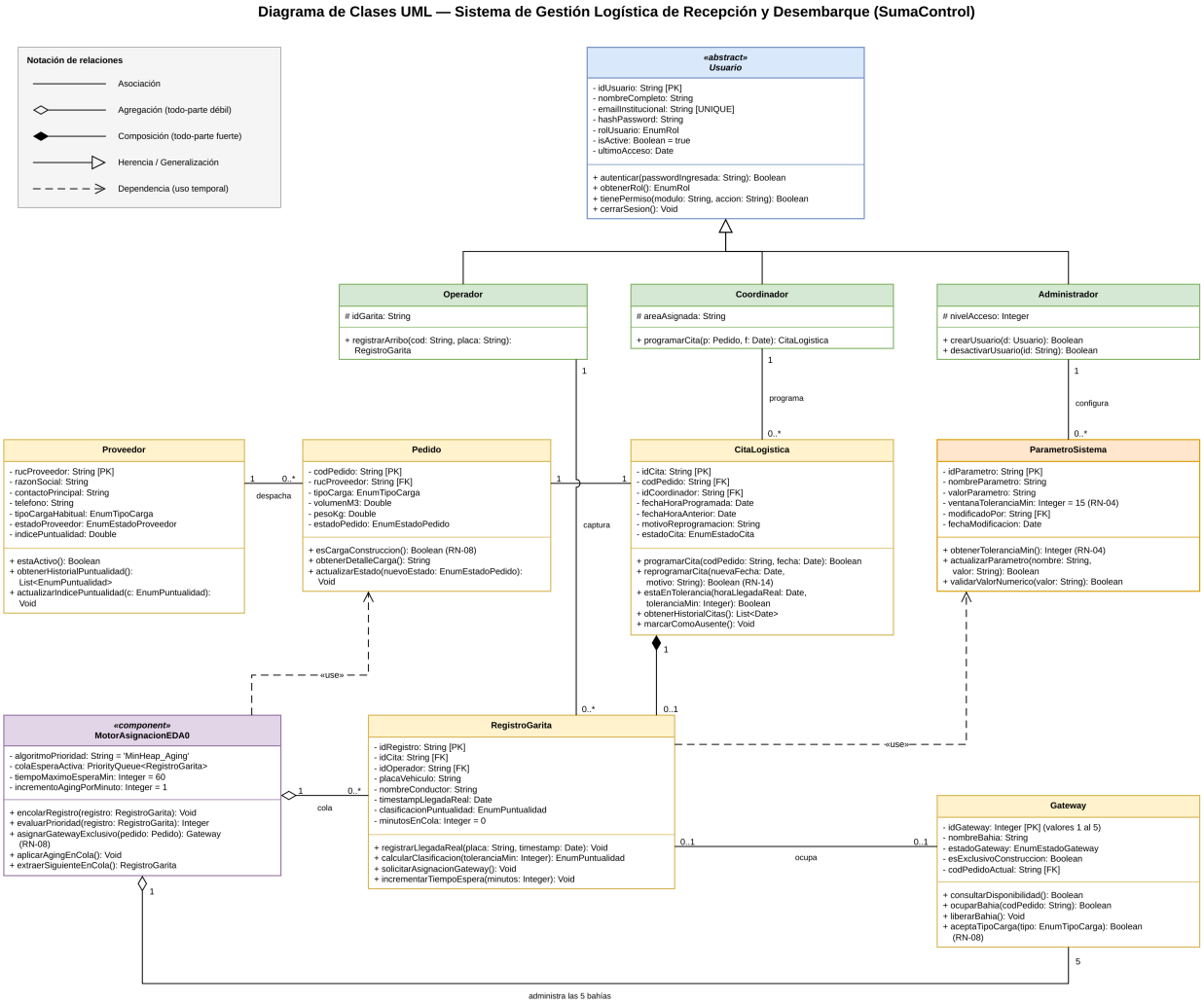


Figura 2. Diagrama de Clases UML — Modelo estructural del dominio (Semana 10).

5.1 Correspondencia entre el modelo y la implementación

La siguiente tabla contrasta cada clase del diagrama contra el componente que la implementa en el backend de PON 0. Es la verificación que exige el Paso 1 de esta actividad, y se presenta sin maquillar: tres clases del modelo no tienen una correspondencia uno a uno con el código, y dos clases del código no figuran en el diagrama.

Clase del diagrama	Responsabilidad	Implementación en PON 0	Observación
«abstract» Usuario Operador · Coordinador Administrador	Identidad y permisos. La jerarquía modela los tres roles del sistema.	`models/Usuario.ts` `middlewares/verificarToken.ts` `middlewares/autorizarRoles.ts`	La herencia del modelo se implementa como un campo rolUsuario más un middleware de autorización, no como tres colecciones. Es la traducción habitual de una jerarquía a una base documental.
Proveedor	Empresa que entrega mercadería. Define la categoría de carga que maneja.	`models/Proveedor.ts` `controllers/proveedorController.ts`	Correspondencia directa.
Pedido	Entrega concreta de un proveedor, con su tipo de producto y su estado.	`models/Pedido.ts` `controllers/pedidoController.ts`	Correspondencia directa. El enum de estados creció respecto del modelo: incorpora EN COLA, DESCARGANDO y FINALIZADO del ciclo del Sprint 3.
CitaLogistica	Franja horaria reservada para un pedido, con su historial de reprogramaciones.	`Campos inicioVentana y finVentana de Pedido` `services/ventanaHoraria.ts`	Divergencia deliberada: la cita no se implementó como colección aparte sino como la ventana del propio pedido. Un pedido tiene exactamente una cita vigente, de modo que separarlos habría obligado a un join en cada consulta sin agregar información.
RegistroGarita	Constancia del arribo real: placa, hora de llegada y clasificación de puntualidad.	`Campos fechaHoraLlegadaReal y puntualidad de Pedido` `services/puntualidad.ts` `controllers/llegadaController.ts`	Divergencia deliberada, por la misma razón: el registro de garita es cardinalidad 1:1 con el pedido. La clasificación sí vive en un servicio propio y aislado.
Gateway	Bahía física de descarga. Cinco en total, la 5 exclusiva de construcción.	`models/Gateway.ts` `controllers/gatewayController.ts`	Correspondencia directa. La regla RN-07/RN-08 se implementa en el modelo mediante tipoCargaEsperado(), no en el controlador, para que la respeten también los seeds.
«component» MotorAsignacionEDA0	Selecciona el siguiente pedido de la cola y le asigna una bahía compatible.	`services/descargas.ts (iniciarDescargaDePedido, promoverPrimeroDeLaCola)`	Implementado parcialmente: la selección por compatibilidad y la promoción automática funcionan; el Heap con aging es el entregable de EDA 0 y no está integrado. Ver sección 7.3.
ParametroSistema	Configuración operativa editable sin volver a desplegar: horario y tolerancias.	`models/Parametro.ts` `services/configuracionOperativa.ts`	Correspondencia directa, con un agregado: la configuración se lee con caché de 60 segundos y valores por defecto si falta un parámetro.
— (sin clase en el modelo)	Registro de cada descarga: bahía, pedido, operador y tiempos de inicio y fin.	`models/Descarga.ts` `controllers/descargaController.ts`	Clase emergente del Sprint 3, posterior al diagrama de la Semana 10. Se incorporará al modelo en la próxima revisión.
— (sin clase en el modelo)	Bitácora inmutable de eventos del sistema para auditoría.	`models/Evento.ts` `services/eventos.ts`	Clase emergente. Da soporte al atributo de calidad RNF-03 (trazabilidad).

Criterio del equipo: las divergencias de CitaLogistica y RegistroGarita son decisiones de diseño, no omisiones: ambas mantienen cardinalidad 1:1 con el pedido, de modo que separarlas en colecciones propias habría obligado a un join en cada consulta sin agregar información. Las clases Descarga y Evento, en cambio, sí son deuda del modelo: surgieron en el Sprint 3, después de que el diagrama se cerrara en la Semana 10, y deben incorporarse en la próxima revisión. Distinguir una cosa de la otra es lo que vuelve útil a esta tabla.

6. VISTA DINÁMICA: DIAGRAMAS DE SECUENCIA

Los dos Casos de Uso nucleares del proceso logístico se modelaron como Diagramas de Secuencia. Ambos respetan la Regla de Oro de trazabilidad nominal: todo mensaje entrante corresponde a un método público declarado en el Diagrama de Clases.

6.1 DS-01 — Registrar llegada real en garita y clasificar puntualidad (CU-02)

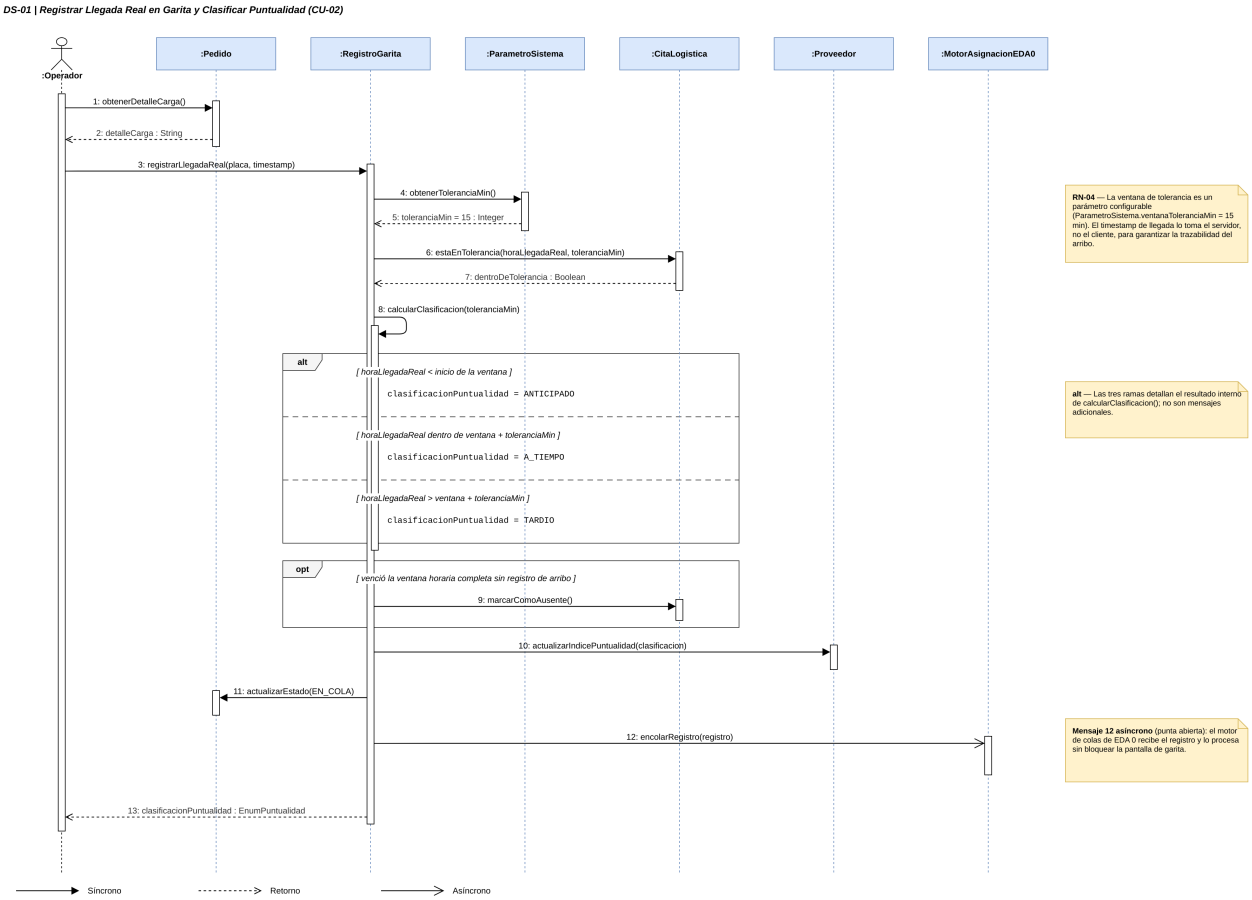


Figura 3. Diagrama de Secuencia DS-01 — CU-02 Registrar llegada y clasificar puntualidad.

Flujo resumido

Paso	Emisor → Receptor	Mensaje	Regla o efecto
1-2	:Operador → :Pedido	obtenerDetalleCarga()	Recupera el tipo de carga del pedido.
3	:Operador → :RegistroGarita	registrarLlegadaReal(placa, timestamp)	RN-05: el timestamp lo fija el servidor.
4-5	:RegistroGarita → :ParametroSistema	obtenerToleranciaMin()	RN-04: la tolerancia es un parámetro configurable, no una constante.
6-7	:RegistroGarita → :CitaLogistica	estaEnTolerancia(horaLlegadaReal, toleranciaMin)	Compara el arribo contra el inicio de la ventana.
8	:RegistroGarita → sí mismo	calcularClasificacion(toleranciaMin)	Fragmento alt con las tres ramas de clasificación.

Paso	Emisor → Receptor	Mensaje	Regla o efecto
9	:RegistroGarita → :CitaLogistica	marcarComoAusente()	Fragmento opt: RN-06, solo si venció la ventana sin arribo.
10	:RegistroGarita → :Proveedor	actualizarIndicePuntualidad(clasificacion)	Alimenta el indicador de cumplimiento del proveedor.
11	:RegistroGarita → :Pedido	actualizarEstado(EN_COLA)	El pedido queda elegible para recibir bahía.
12	:RegistroGarita → :MotorAsignacionEDA0	encolarRegistro(registro)	Mensaje asíncrono: no bloquea la pantalla de garita.
13	:RegistroGarita → :Operador	clasificacionPuntualidad : EnumPuntualidad	Retorno al actor con la clasificación resultante.

6.2 DS-02 — Asignar gateway y gestionar cola de prioridad (CU-03)

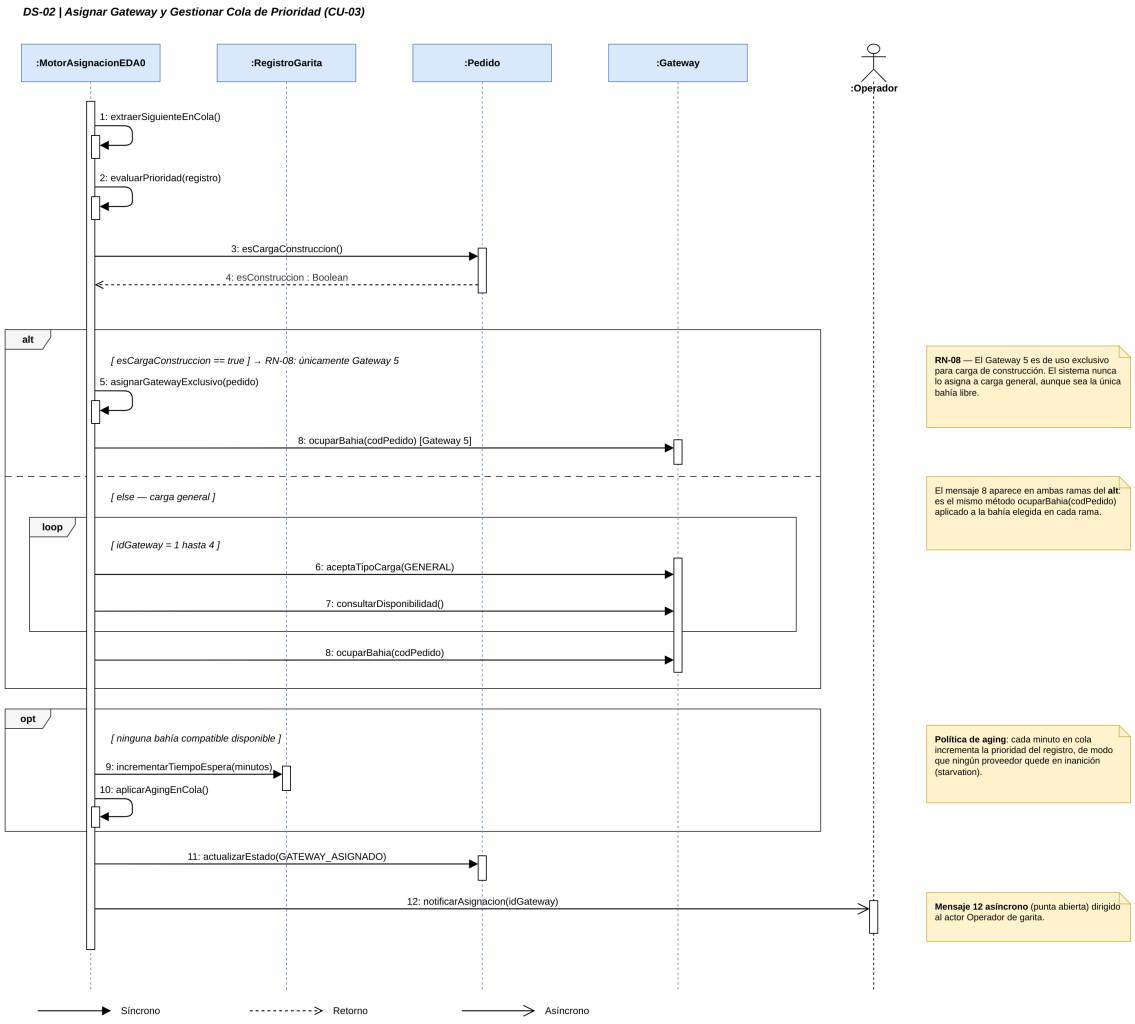


Figura 4. Diagrama de Secuencia DS-02 — CU-03 Asignar gateway y gestionar cola.

Flujo resumido

Paso	Emisor → Receptor	Mensaje	Regla o efecto
1-2	:MotorAsignacionEDA0 → sí mismo	extraerSiguienteEnCola() · evaluarPrioridad()	Selecciona al siguiente de la cola.
3-4	:MotorAsignacionEDA0 → :Pedido	esCargaConstruccion()	Determina qué rama del fragmento alt aplica.
5	:MotorAsignacionEDA0 → sí mismo	asignarGatewayExclusivo(pedido)	Rama de construcción: RN-08, únicamente Gateway 5.
6-7	:MotorAsignacionEDA0 → :Gateway	aceptaTipoCarga(GENERAL) · consultarDisponibilidad()	Fragmento loop sobre las bahías 1 a 4.
8	:MotorAsignacionEDA0 → :Gateway	ocuparBahia(codPedido)	Mismo método en ambas ramas, sobre la bahía elegida.

Paso	Emisor → Receptor	Mensaje	Regla o efecto
9-10	:MotorAsignacionEDA0 → :RegistroGarita	incrementarTiempoEspera() · aplicarAgingEnCola()	Fragmento opt: sin bahía compatible disponible. Ver la sección 7.3.
11	:MotorAsignacionEDA0 → :Pedido	actualizarEstado(GATEWAY_ASIGNADO)	RN-11: cambio de estado al asignarse la bahía.
12	:MotorAsignacionEDA0 → :Operador	notificarAsignacion(idGateway)	Mensaje asíncrono dirigido al actor.

7. REGLAS DE NEGOCIO Y COHERENCIA CON LA ARQUITECTURA

Esta sección responde al Paso 1 de la actividad: verificar la coherencia y la trazabilidad entre las reglas de negocio RN-01 a RN-14 y los componentes de arquitectura del centro logístico.

El SRS de la Semana 7 formaliza explícitamente tres reglas —RN-04, RN-08 y RN-14— por ser las que se modelan como flujos de extensión en los Casos de Uso. El resto del comportamiento quedaba expresado en los criterios de aceptación de las Historias de Usuario sin un identificador propio. Para esta consolidación se completó el catálogo hasta RN-14, derivando cada regla de dos fuentes propias del equipo: los criterios de aceptación en sintaxis Gherkin refinados en la Semana 8 y las validaciones efectivamente implementadas en el backend. No se incorporó ninguna regla que el sistema no aplique: el catálogo describe el sistema que existe, no uno deseable.

Cada fila recorre el camino completo desde la regla hasta la prueba que la respalda, pasando por la clase que la modela y el componente que la implementa.

RN	Enunciado de la regla	Clase o componente UML	Implementación	Evidencia
RN-01	Toda cita debe caer dentro del horario operativo configurable y en día hábil.	CitaLogistica · ParametroSistema	`services/ventanaHoraria.ts` `POST /api/pedidos`	PU-10, PU-11 CP-11
RN-02	Dos citas no pueden solaparse; dos turnos consecutivos sí son válidos.	CitaLogistica	`services/ventanaHoraria.ts` `(seSolapan)`	PU-12 CP-11
RN-03	El número de pedido se autogenera y su correlativo no se reutiliza.	Pedido	`services/numeroPedido.ts` `POST /api/pedidos`	PU-15 a PU-18 CP-12
RN-04	La tolerancia de puntualidad es configurable; la clasificación tiene cuatro estados.	RegistroGarita · ParametroSistema	`services/puntualidad.ts` `POST /api/llegadas`	PU-01 a PU-05 CP-01, CP-03
RN-05	El timestamp de arribo lo fija el servidor, no el cliente.	RegistroGarita	`controllers/llegadaController.ts`	CP-01
RN-06	Vencido el límite sin llegada, el pedido pasa a AUSENTE automáticamente.	RegistroGarita	`services/controlAusencias.ts` `POST /api/llegadas/control-ausencias`	CP-04
RN-07	Las bahías 1 a 4 admiten únicamente carga general.	Gateway	`models/Gateway.ts` `(tipoCargaEsperado)`	PU-07, PU-08 CP-01, CP-06
RN-08	El Gateway 5 es exclusivo de carga de construcción.	Gateway	`models/Gateway.ts` `services/descargas.ts`	PU-06, PU-07 CP-05, CP-06, CP-08
RN-09	Una bahía solo se ocupa si está LIBRE; la ocupación es atómica.	Gateway · MotorAsignacionEDA0	`services/descargas.ts` `(findOneAndUpdate condicional)`	CP-07
RN-10	Sin bahía compatible libre, el pedido permanece en cola.	MotorAsignacionEDA0	`services/descargas.ts`	CP-02
RN-11	Al finalizar, la bahía se libera y promueve al siguiente de la cola.	MotorAsignacionEDA0 · Gateway	`services/descargas.ts` `POST /api/descargas/finalizar`	CP-07, CP-08
RN-12	La cola se atiende por antigüedad de arribo real.	MotorAsignacionEDA0	`services/descargas.ts` `(promoverPrimeroDeLaCola)`	CP-07, CP-10
RN-13	Una bahía FUERA DE SERVICIO no recibe asignaciones.	Gateway	`services/descargas.ts` `PUT /api/gateways/:id`	CP-09

RN	Enunciado de la regla	Clase o componente UML	Implementación	Evidencia
RN-14	La reprogramación deja constancia de la ventana anterior.	CitaLogistica	`controllers/pedidoController.ts` `PUT /api/pedidos/:id/reprogramar`	PU-13, PU-14 CP-11

7.1 Resultado de la verificación

Las catorce reglas tienen una clase o componente que las modela y un componente que las implementa. Trece de las catorce cuentan además con al menos un caso de prueba ejecutado que las verifica; la excepción es RN-12, que se verifica en su comportamiento observable pero no en su característica de eficiencia algorítmica, por el motivo que se explica en 7.3.

7.2 Inconsistencias detectadas y resueltas

La consolidación dejó al descubierto dos desajustes entre el modelo y la implementación, ambos ya descritos en la sección 5.1: las clases Descarga y Evento existen en el código y no en el Diagrama de Clases, y las clases CitaLogistica y RegistroGarita se implementaron como campos del pedido en lugar de colecciones propias. El primero es deuda de modelado y se corrige en la próxima revisión del diagrama; el segundo es una decisión de diseño que el equipo sostiene y documenta.

Se detectó además una inconsistencia en el propio modelo dinámico: los mensajes 9 y 10 de DS-02 invocan aplicarAgingEnCola() e incrementarTiempoEspera(), métodos que el Diagrama de Clases declara en MotorAsignacionEDA0 y RegistroGarita pero que el backend de PON 0 todavía no implementa. El diagrama es correcto respecto del diseño acordado; lo que falta es la integración descrita abajo.

7.3 Alcance de EDA 0: la cola de prioridad con envejecimiento

El atributo de calidad RNF-08 de la Historia HU-003 establece que la cola de prioridad se implementa como un Heap binario con política de envejecimiento, en C++, dentro de la asignatura EDA 0. Ese componente es el que el Diagrama de Clases modela como «component» MotorAsignacionEDA0.

En la Versión Beta Funcional que documenta este avance, el backend de PON 0 resuelve el orden de atención mediante una consulta ordenada por hora de llegada real, delegando el ordenamiento en el motor de la base de datos. El criterio es el mismo que persigue la política de envejecimiento —atender primero a quien lleva más tiempo esperando— pero la estructura de datos que lo implementa todavía no es el Heap de EDA 0.

Por eso RN-12 se declara verificada en su comportamiento (CP-07 y CP-10 comprueban que la cola se atiende por antigüedad de arribo) y pendiente en su eficiencia: medir una complejidad O(log n) sobre un ordenamiento que la base resuelve de otro modo sería reportar un resultado que la implementación no respalda. La trazabilidad con el entregable de EDA 0 queda abierta y se cerrará cuando ambos componentes se integren.

8. GOBERNANZA DEL REPOSITORIO Y ESTADO DEL INCREMENTO

Artefacto	Referencia
Repositorio oficial del equipo	`https://github.com/202401579-hue/Proyecto-Integrador`
Etiqueta de liberación Beta	`v2.0.0-beta` `https://github.com/202401579-hue/Proyecto-Integrador/releases/tag/v2.0.0-beta`
Pull Request con Code Review	`https://github.com/202401579-hue/Proyecto-Integrador/pull/21` `Aprobado por Victor5977`
Diagramas UML versionados	`docs/diagramas/ (casos de uso, clases y secuencia, en PNG y .drawio)`
Suites de prueba	`backend/src/__tests__/ (48 casos unitarios)` `backend/src/pruebas/ (12 casos funcionales)`

Nivel de prueba	Casos	Resultado
Unitarias (Jest)	48	48 aprobadas
Funcionales sobre la API REST	12	12 aprobados
Reglas de negocio cubiertas	RN-01 a RN-14	14 de 14 trazadas

8.1 Declaración de uso de Inteligencia Artificial (estándar FEEN)

📌 **DECLARACIÓN OBLIGATORIA:** para la elaboración de este informe consolidado el equipo utilizó las herramientas de Inteligencia Artificial Claude y ChatGPT con tres propósitos acotados: (1) guiar mediante preguntas el contraste entre los diagramas UML y el código fuente; (2) apoyar la redacción y la ortografía del documento; y (3) organizar en tablas la información ya producida por el equipo en las Semanas 7 a 11. La elicitación de requisitos, la construcción de los diagramas, la decisión de sostener las divergencias de diseño de CitaLogistica y RegistroGarita, el reconocimiento de Descarga y Evento como deuda de modelado y la decisión de no reportar métricas de eficiencia sobre un componente no integrado fueron realizadas y validadas críticamente por los integrantes del Equipo Scrum, que asumen la responsabilidad técnica completa de lo aquí consignado, de conformidad con los Lineamientos del Uso de la Inteligencia Artificial de la FEEN / UJMD.